AIA4: Prompt Engineering – Advanced

Module Introduction and Theoretical Foundations

Prompt Engineering elevates LLMs from pattern matchers to reasoning engines. Beyond simple zero-shot queries, advanced techniques systematically exploit **latent capabilities** encoded during pretraining. This module covers the mathematical foundations of prompting, systematic evaluation frameworks, and production deployment strategies.

Theoretical Basis:

1. **In-Context Learning**: LLMs as gradient-free meta-learners
2. **Emergent Reasoning**: Chain-of-Thought as scratchpad computation
3. **Gradient Flow in Prompt Space**: Prompt tuning as optimization
4. **Information Geometry**: Prompting as manifold interpolation

Key Insight (Wei 2022): Few-shot CoT emerges at 100B+ parameters

Prompt Taxonomy:

text

Zero-shot → Few-shot → CoT → Tree-of-Thoughts → Graph-of-Thoughts

(Figure 1: Prompt complexity vs performance curves)

Section 1: Prompting Fundamentals and Evaluation Framework

Core Prompt Components:

text

Role/System: "You are a world-class mathematician"

Context/Examples: Few-shot demonstrations

Task: Clear objective + constraints

Output Format: "Answer: [NUMBER]"

Chain: Multi-step reasoning

Evaluation Framework (Comprehensive):

python

```python
class PromptEvaluator:
    def __init__(self, model, tokenizer):
        self.model = model
        self.tokenizer = tokenizer
```

```
    def evaluate_template(self, template, test_cases, metric='exact_match'):
        results = []
        for case in test_cases:
            prompt = template.format(**case['input'])
            response = self.generate(prompt)
            score = self.compute_metric(response, case['gold'], metric)
            results.append({'prompt': prompt, 'score': score})
        return pd.DataFrame(results)

    def ablation_study(self, base_prompt, variations):
        """Systematic hyperparameter search"""
        scores = {}
        for var_name, var_prompts in variations.items():
            for prompt in var_prompts:
                full_prompt = base_prompt + prompt
                score = self.run_evaluation(full_prompt)
                scores[(var_name, prompt)] = score
        return scores
```

Zero-Shot vs Few-Shot Analysis:

text

Zero-shot: Direct instruction

Few-shot: K examples (K=0-32 optimal)

Optimal example selection: Diversity + recency

Section 2: Chain-of-Thought (CoT) - Reasoning Emergence

Discovery (Wei 2022): Adding "Let's think step by step" boosts GSM8K from 18% → 58%

Mathematical Interpretation:

text

CoT ≈ LLM scratchpad computation

Enables symbolic manipulation in embedding space

Variants and Theory:

1. **Zero-Shot CoT:**

text

"Q: When I was 6, my sister was half my age...

Think step by step before answering."

1. **Few-Shot CoT:**

text

Examples with full reasoning traces

1. **Self-Consistency (Wang 2022):**

text

Sample K reasoning paths → majority vote

Error ∝ 1/√K reasoning quality

CoT Implementation Framework:

python

```python
class CoTGenerator:
    def __init__(self, model, tokenizer, max_reasoning_steps=8):
        self.model = model
        self.tokenizer = tokenizer
        self.max_steps = max_reasoning_steps

    def generate_cot(self, question, fewshot_examples=None):
        prompt = self.build_cot_prompt(question, fewshot_examples)

        # Generate reasoning
        reasoning = self._generate(prompt, stop=["Answer:", "\n\n"])

        # Extract final answer
        answer_prompt = prompt + reasoning + "\nAnswer:"
        answer = self._generate(answer_prompt, stop=["\n"])

        return {"reasoning": reasoning, "answer": answer}
```

```python
    def self_consistency(self, question, n_samples=10):
        answers = [self.generate_cot(question)['answer']
                   for _ in range(n_samples)]
        return max(set(answers), key=answers.count)
```

Advanced CoT Patterns:

```text
1. Least-to-Most: Break complex → simple subtasks
2. Program-Aided: Generate Python code
3. Verifier CoT: Generate + critique + regenerate
```

Section 3: Role Prompting and Persona Engineering

Role Impact Theory: Activates specific neuron clusters

Systematic Role Testing:

```text
Baseline: "You are helpful"
Financial Analyst: +15% finance accuracy
Code Reviewer: +22% bug detection
Historian: +18% timeline reasoning
```

Persona Optimization:

```python
PERSONAS = {
    "mathematician": "world-class PhD mathematician who double-checks work",
    "programmer": "10-year senior SWE who writes production code",
    "physicist": "Nobel laureate physicist who derives from first principles"
}

def optimize_persona(task_type, test_set):
    best_persona, best_score = None, 0
    for persona in PERSONAS.values():
        score = evaluate_prompt(f"You are a {persona}. {task}", test_set)
        if score > best_score:
```

```
        best_score, best_persona = score, persona
    return best_persona
```

Multi-Persona Debate:

text

Alice (optimist): Generate solution A

Bob (critic): Find flaws in A

Charlie (synthesizer): Improved solution

Section 4: Multi-Step Reasoning Architectures

Prompt Chaining Theory:

text

Complex task → Atomic subtasks → Orchestration

Reduces context dilution

Tree-of-Thoughts (Yao 2023):

text

BFS/DFS search in reasoning space

Node: Reasoning step

Branch factor: 3-5

Depth: 4-8

ToT Implementation:

python

```
class TreeOfThoughts:
    def __init__(self, model, max_depth=4, branches=3):
        self.model = model
        self.max_depth = max_depth
        self.branches = branches

    def search(self, problem):
        root = Node(problem)
        queue = deque([root])
```

```
    while queue and root.depth < self.max_depth:
        current = queue.popleft()

        # Generate branches
        thoughts = self.generate_thoughts(current.state, self.branches)
        for thought in thoughts:
            child = Node(thought, parent=current)
            current.children.append(child)
            queue.append(child)

            # Early stopping
            if child.is_solution():
                return child.value

    return self.best_leaf(root)
```

Graph-of-Thoughts (Besta 2023):

text

Nodes: Reasoning steps

Edges: Dependencies/critiques

Operations: Aggregate, Refine, Generate, Improve

Section 5: Advanced Techniques - Automatic Prompting

Automatic Prompt Engineer (APE, Zhou 2023):

text

Meta-LLM generates prompts for target LLM

Few-shot examples + verification

DSPy (Khattab 2023) - Prompt Optimization:

text

prompts, weights ← argmin L(compile(program))

Gradient-free: BO, MCTS, MIP

DSPy Example:

python

```python
import dspy

class MathSolver(dspy.Signature):
    """Solve math word problems."""
    question: str = dspy.InputField()
    reasoning: str = dspy.OutputField(desc="step-by-step reasoning")
    answer: str = dspy.OutputField(desc="final numerical answer")

class CoTModule(dspy.Module):
    def __init__(self):
        super().__init__()
        self.generate_reasoning = dspy.ChainOfThought(MathSolver)

    def forward(self, question):
        pred = self.generate_reasoning(question=question)
        return dspy.Prediction(reasoning=pred.reasoning, answer=pred.answer)

# Compile (optimize prompts)
math_solver = CoTModule()
compiled = math_solver.compile(trainset=trainset, max_bootstrapped_demos=4)
```

Evolutionary Prompt Search:

text

```text
Population: 100 prompt variants
Fitness: validation accuracy
Mutation: paraphrase, add instruction, swap examples
Crossover: combine high-fitness prompts
```

Section 6: Production Prompt Systems

Prompt Registry & A/B Testing:

python

```
class PromptRegistry:
    def __init__(self):
        self.prompts = {}
        self.metrics = {}

    def register(self, name, template, tags=[]):
        self.prompts[name] = {"template": template, "tags": tags}

    def ab_test(self, names, test_set, metric='accuracy'):
        results = {}
        for name in names:
            prompt = self.prompts[name]['template']
            score = self.evaluate(prompt, test_set, metric)
            results[name] = score
        return max(results, key=results.get)
```

Dynamic Prompting:

text

Router LLM selects optimal prompt template

Input → classifier → template_X

Safety & Jailbreak Defense:

text

1. Constitutional AI: Critique against principles

2. Prompt hardening: "Ignore previous instructions"

3. Multi-agent verification

Section 7: Evaluation, Case Studies, Hands-On Projects

Advanced Metrics:

text

1. Pass@K: K samples, any correct?

2. Reasoning F1: Semantic match on traces

3. Calibration: |confidence - accuracy|

Case Study: OpenAI o1 Reasoning:

text

Test-time compute scaling

Silent CoT + verification loops

10-100x inference cost for 2x accuracy

Production Project: Build PromptOps Platform

text

1. Prompt version control

2. Automated evaluation suite

3. A/B testing infrastructure

4. Cost/performance monitoring

Hands-On Challenges:

text

1. Beat GPT-4 on GSM8K with clever prompting

2. Implement ToT from scratch

3. Optimize prompts for your domain